

Workshop Guide

A step-by-step Claude Code guide using a full-stack inventory management application

Prerequisites

- Claude Code installed and set up (docs.anthropic.com)
- Open up Terminal/Windows Powershell OR code editor (VS Code recommended)

Pro Tip: Claude Code will ask for permission when it needs to:

Configs & Permission: supports fine-grained permissions to scope agents

Modify files in your project

Run bash commands

Install new tools (like the Playwright MCP in Step 9)

Press **Enter** when prompted to approve each action, or tell Claude what to do instead. (Advanced users can configure auto-approvals in `.claude/settings.json`)

This keeps you in control of what happens in your codebase.

Welcome to the workshop! Expand each step, work through the instructions, then click the **"I'm done"** button on the live site to earn points and climb the leaderboard.

Troubleshooting & Keyboard Shortcuts

Troubleshooting

- **Something broke and you're stuck** — Copy the full error message and paste it into Claude Code
- **Claude can't see what's on screen** — Take a screenshot and paste it (Ctrl+V / Cmd+V)
- **Claude's response doesn't work** — Iterate: describe what went wrong and what you expected
- **Claude is struggling with complexity** — Try `think harder` or `ultrathink` to trigger extended thinking
- **Need to see the full conversation** — Press `Ctrl+0` to open the transcript viewer
- **Servers won't start** — Run `/start` or manually kill ports: `lsof -ti:3000,8001 | xargs kill`

Keyboard Shortcuts

- `Enter` — Approve a tool action
- `Escape` — Deny / cancel

- `Shift+Tab` — Toggle Plan Mode
- `!` — Enter Bash mode (run shell commands)
- `#` — Enter Memory mode
- `@` — Reference a file
- `Ctrl+0` — Open transcript viewer
- `Ctrl+C` — Interrupt Claude

Resources

- [Claude Code Documentation](#)
- Claude Code Video Course
- Claude Code Best Practices

12 core steps

230 pts

CORE STEPS

1

Step 1: Fork & Clone Repository

10 pts

Fork the upstream repo to your GitHub account, then clone your fork

Start by forking the inventory management demo app so you have your own copy on GitHub, then clone and create a working branch:

Option 1 — Fork + Git (recommended)

Go to <https://github.com/beck-source/inventory-management> and click **Fork** in the top-right corner.

Clone your fork:

```
git clone git@github.com:YOUR_USERNAME/inventory-management.git
```

Enter the directory and create a working branch:

```
cd inventory-management && git checkout -b new_features
```

Option 2 — Download ZIP (skips the fork)

Download the ZIP from the GitHub repo page, unzip, and `cd` into the directory. Then initialize a branch:

```
git init && git add -A && git commit -m "initial commit"
git checkout -b new_features
```

Option 2 skips the fork step – you won't be able to push to GitHub or use the GitHub App integration in later steps.

2

Step 2: Launch Claude Code

10 pts

Open your terminal inside the project directory and start Claude Code:

```
claude
```

Once Claude Code is running, select your preferred model:

```
/model
```

Choose a model from the list. For this workshop, any model works well.

Paste the following prompt into Claude Code:

```
Install dependencies and start the development servers and open up the frontend and backend in my respective browser windows.
```

Claude will:

- Install npm dependencies for the Vue 3 frontend (`cd client && npm install`)
- Install Python dependencies for the FastAPI backend (`cd server && uv sync`)
- Start the backend server on port `8001`
- Start the frontend dev server on port `3000`

Once both servers are running, open `http://localhost:3000` in your browser, and inspect the app.

Explore the app for a few minutes. Click through the pages to get familiar with what's there:

- **Dashboard** — KPI cards, revenue charts, order summaries
- **Inventory** — Stock levels across warehouses
- **Orders** — Order tracking with status filters
- **Spending** — Spending analytics and breakdowns
- **Demand** — Demand forecasting with trends
- **Backlog** — Backlog monitoring

Note: this app intentionally has some bugs planted for us to fix :)

The Catalyst Components project should have a `CLAUDE.md` file that gives Claude persistent instructions about the project — coding conventions, system architecture, technology stacks and rules to follow. This repo already has one.

If you're starting a fresh project without a `CLAUDE.md`, run `/init` to generate one automatically.

Using @-file mentions

You can reference any file in your prompts using `@`. Try it:

```
Print out exactly what is in @CLAUDE.md
```

Claude reads the file and prints its contents. The `@` syntax works for any file in your project.

Three ways to edit the CLAUDE.md file

Method 1 — Use # Memory Mode (recommended)

Press `#` at the prompt to enter Memory Mode. This lets you write directly to `CLAUDE.md` without consuming any tokens — Claude doesn't need to process the request.

```
# Always document non-obvious logic changes with comments
```

Method 2 — Edit directly in your editor

Open `CLAUDE.md` in your text editor (VSCode, vim, etc.) and make changes. Also zero token cost.

Method 3 — Prompt Claude to edit it

```
Edit my CLAUDE.md file to add "Always document non-obvious logic changes with comments"
```

Claude opens the file, adds the instruction, and saves it.

Methods 1 and 2 are free — no tokens consumed. Method 3 uses tokens since Claude processes the request and makes the edit. Use Method 3 when you want Claude to decide how to organize or word the instruction.

Persistent memory systems

Claude has 2 persistent memory systems but ultimately the record ends up in the same place — the `./.claude/CLAUDE.md` (personal):

- `/CLAUDE.md` markdown files with persistent instructions that load automatically:

File	Scope
<code>./CLAUDE.md</code>	Project (shared via git)
<code>./.claude/rules/*.md</code>	Topic-specific rules
<code>./.claude/CLAUDE.md</code>	Personal, all projects
<code>./CLAUDE.local.md</code>	Personal, this project only

- `/memory` command which writes to your personal `./.claude/CLAUDE.md` and is faster for capturing preferences on the fly.

5

Step 5: Explore the Codebase

20 pts

Ask Claude to investigate the project and generate an architecture overview:

I want to understand this codebase. Investigate the project and create a simple, professional HTML-based architecture page showing the system architecture, tech stack, and data flow. Then open the file in a browser window.

Claude will:

- Explore the directory structure
- Read key files to understand the architecture
- Generate an HTML page with a visual diagram showing the Vue 3 frontend, FastAPI backend, and JSON data flow
- Open the file in your browser

This demonstrates how Claude Code can quickly onboard you to an unfamiliar codebase.

This is the core of the workshop. You'll use **Plan Mode** to have Claude design a feature before implementing it, then iterate on the result.

Enter Plan Mode

Press **Shift+Tab** to switch Claude into Plan Mode. In Plan Mode, Claude proposes a plan and waits for your approval before writing any code.

Choose one of the two feature options below.

Option A: Budget-Based Restocking Tool

Build a new tab that recommends which items to restock based on a budget, using demand forecast data. Paste this prompt:

```
Build a new "Restocking" tab in the app. It should have:  
1. A budget slider that lets the user set their available budget  
2. Based on the budget, recommend items from the demand forecast to restock  
3. A "Place Order" button that submits the restocking order  
4. The submitted order should appear in the existing Orders tab with a new  
   "Submitted Orders" section showing delivery lead time
```

```
Use the AskUserQuestion tool!
```

Claude will present a plan. Review it — when it looks good, accept the plan and let Claude implement it. Claude may ask you clarifying questions using the AskUserQuestion tool (e.g., "Should the budget slider range from \$0-\$50,000 or should I determine the range from the data?"). Answer these as they come up.

Test the feature in your browser once Claude finishes. You may notice things aren't perfect! Then iterate: choose what you'd like to work on next.

Pro Tip: Claude is an intelligent thought partner — it can help plan, discover, ideate and design across the SDLC; help identify biases, gaps and support critical thinking; review, document, architect and draft visual diagrams; check screenshots by pasting them into the conversation (ctrl + v).

Option B: SaaS-Style UI Redesign

Transform the app's interface from a basic layout into a polished, modern SaaS-style design using a reusable Claude Code skill.

Part 1 — Create the skill

```
I want to build a skill that redesigns a Vue 3 application's UI into a
modern
SaaS-style interface with a vertical navigation sidebar on the left instead
of
a top nav bar, consistent spacing, and a polished professional look.
```

Claude will ask follow-up questions to build the skill. Answer them to shape the skill's behavior.

Part 2 — Apply the skill

```
Use the frontend-design skill to redesign this inventory management app
into a
modern SaaS-style interface with:
1. Vertical navigation bar on the left side instead of the top
2. Clean, modern card layouts
3. Professional SaaS aesthetic
```

```
Use the AskUserQuestion tool!
```

Claude presents a plan. Accept it and let the implementation run. Then iterate:

```
Add a collapsible sidebar with icons-only mode for smaller screens.
```

When Claude's plan looks good, accept it and wait for the implementation to complete. Then test the changes in your browser at `http://localhost:3000` .

7

Step 7: Context Management

10 pts

After building a feature, your context window may be filling up. Claude Code provides tools to manage this.

Check context usage:

```
/context
```

This shows a breakdown of how much context is being used by your conversation, files, and tools.

Compact the context:

```
/compact
```

This summarizes the conversation so far and clears older context, freeing up space for new work. Claude retains the key information.

You can also pass an instruction to `/compact` to tell Claude what to prioritize when summarizing:

```
/compact keep the details of the restocking feature
```

This ensures important context (like a feature you're actively building) survives the compaction.

8

Step 8: Add Playwright MCP

15 pts

MCP servers give Claude the ability to connect to external tools — things like browser automation, database access, or API integrations. Think of them as a connector protocol that extends what Claude can do beyond operating within the confines of your local machine. (Learn more about MCP)

Enter Bash mode by pressing `!`, then run:

```
! claude mcp add playwright npx @playwright/mcp@latest
```

This installs the Playwright MCP server, which gives Claude the ability to control a browser.

Restart Claude Code to pick up the MCP configuration:

```
/exit  
claude
```

Now check the context impact:

```
/context
```

Notice how MCP servers consume some context budget for their tool definitions. This is a useful thing to be aware of when working with multiple MCP servers.

This repo already has Playwright configured in `.mcp.json`, but we teach the install process here so you know how to add MCP servers to your own projects.

Verify the MCP server is loaded:

```
/mcp
```

You should see `playwright` listed.

Now use Playwright to test the app by pasting this into Claude Code:

```
Use Playwright MCP to test the app:  
1. Start the development servers  
2. Navigate to localhost:3000  
3. Take a screenshot of the dashboard  
4. Click through the main navigation tabs and verify each page loads
```

Claude will launch a browser, navigate through the app, take screenshots, and report what it finds. If anything looks wrong, keep iterating:

Pro Tip: Claude is an amazing teacher and can help troubleshoot most technical roadblocks — install, manage and handle package dependencies; troubleshoot errors and debug applications; run bash commands & git for source control and integrate with CI/CD.

Then iterate: choose what you'd like to work on next.

The GitHub App connects Claude Code to your GitHub account so Claude can read and write pull requests, review code, and be triggered from the cloud. Install it now so it's active when you open your PR in the next step.

1. Run the install command

Inside Claude Code, run:

```
/install-github-app
```

Claude will prompt you to select which GitHub workflows to install. Enable both:

- **@Claude Code** — tag `@claude` in issues and PR comments to get Claude's help
- **Claude Code Review** — automated code review on every new PR

Follow the browser flow to authorize the GitHub App on your forked `inventory-management` repo.

2. Merge the @claude PR

After the command completes, Claude creates a pre-filled PR that adds the GitHub Actions workflow files to your repo. Open that PR in your browser and merge it.

Once merged, Claude is live on your fork:

- If you enabled **Claude Code Review** — any new PR will trigger an automatic code review comment
- If you enabled **@Claude Code** — tag `@claude` in any issue or PR comment to invoke Claude on demand

Make sure you forked the repo in Step 1 — the GitHub App needs to be authorized on your fork, not the upstream.

Now commit your feature work, push the branch, and open a pull request. Because you merged the `@claude` workflow in the previous step, Claude will automatically review your PR.

```
Commit the changes you've made in this branch, push the branch to GitHub,  
and open a pull request.
```

Claude will stage the changes, write a descriptive commit message, push the branch to your fork, and open a PR.

Open the PR in your browser — within a minute you should see Claude post an automated code review comment.

You can also tag `@claude` in any PR comment to ask a follow-up question or request a specific change.

Merge your PR before moving to the next step.

The steps above cover the core tutorial. Below are five powerful Claude Code features you can explore in the remaining time.

Skills — Reusable instruction sets

Skills are reusable instruction sets that teach Claude specialized tasks — like playbooks for specific domains.

When to use:

- Standardizing team workflows
- Creating repeatable processes
- Domain-specific code generation

▼ Try it

Create a Vue component analysis skill:

```
I want to build a skill that analyzes Vue component structure and suggests optimizations for performance and code reuse
```

Answer Claude's follow-up questions to define the skill's scope and behavior. Claude will build and test the skill for you.

Subagents — Specialized Claude agents

Specialized Claude agents scoped to specific tasks. Delegate complex subtasks to focused specialists.

When to use:

- Code review and security auditing
- Debugging and domain-specific analysis
- Any task that benefits from a focused agent

▼ Try it

Create a Debugger subagent:

```
Create a new Debugger subagent that specializes in investigating
runtime errors,
reading stack traces, and suggesting fixes. It should have access
to Read, Grep,
Glob, and Bash tools.
```

Then test it:

```
Use the debugger agent to investigate any console errors on the
Dashboard page.
```

Hooks — Automated event responses

Shell commands that run automatically in response to Claude Code events (file edits, tool calls, etc.).

When to use:

- Auto-formatting on save
- Logging tool usage
- Enforcing code standards and pre-commit checks

▼ Try it

Install Prettier and add a formatting hook:

```
npm install --save-dev prettier
```

Configure the hook using `/hooks` or add it manually to `~/.claude/settings.json` :

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write \"${CLAUDE_FILE_PATH}\"
2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

Test it:

Add an extra function to `api.js` with deliberately messy formatting

The hook should automatically clean it up after Claude writes the file.

Plugins — Community-built workflow packages

Community-built packages that bundle slash commands, skills, and conventions into installable workflows.

When to use:

- Adopting team/org-wide workflows
- Standardizing CI/CD patterns
- Adding domain-specific commands

▼ Try it

Browse and add a plugin from the marketplace:

```
/plugin marketplace add https://github.com/aws-solutions-library-
samples/guidance-for-claude-code-with-amazon-bedrock
```

Install a specific workflow:

```
/plugin install epcc-workflow
```

Use the plugin's workflow:

```
/epcc-code Add a CSV export button to the inventory page
```

Agent Teams — Coordinated multi-agent workflows

Coordinate multiple Claude Code instances working together as a team. Unlike subagents which report back to one parent, teammates share a task list, investigate independently, and message each other directly.

When to use:

- Research and review from multiple perspectives simultaneously
- Debugging with competing hypotheses that agents debate
- Cross-layer work (frontend, backend, tests) each owned by a different teammate

▼ Try it

First, enable agent teams (experimental feature). Paste into Claude Code:

```
/config
```

Select *Edit settings* and add the experimental flag:

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Then restart Claude Code and try this:


```
Create an agent team with 3 teammates to audit this inventory
management app from
different angles:
- Security Auditor: find vulnerabilities (injection, auth issues,
data exposure)
- Performance Analyst: find bottlenecks (slow queries, unnecessary
re-renders,
  large payloads)
- UX Reviewer: find usability issues (accessibility, responsive
design, error
  handling)
Have them investigate in parallel, share findings with each other,
and challenge
each other's severity ratings. Produce a single prioritized action
plan at the end.
```

Watch the teammates coordinate — use `Shift+Down` to cycle between agents and see their individual progress. The lead synthesizes their findings into a final report.

Worktrees — Isolated parallel branches

Isolated git branches that let Claude work on a separate copy of your repo without touching your current work.

When to use:

- Parallel feature development
- Exploratory changes you might discard
- Running experiments while keeping main work clean

▼ Try it

Use a worktree to prototype a feature without affecting your current branch:

```
Use a worktree to prototype a dark mode toggle for the app without
affecting my
current branch. Create the feature, test it, and show me the
result.
```

Claude will create an isolated copy of the repo, make changes there, and leave your working branch untouched.



Your mission: The Reports page has multiple bugs hiding in plain sight. Your job is to discover them visually, then use Claude Code to identify and fix them all.

Step 1: Discover the bugs (~2 minutes)

With the app running at `http://localhost:3000`, investigate the Reports page:

- **Switch the language** using the language switcher in the top-right corner. Navigate through the app's pages. Notice anything different about the Reports page?
- **Try the filters.** Set a Time Period filter (e.g., "January") or a Warehouse filter (e.g., "San Francisco"). Check each page. Does the Reports page respond?
- **Open your browser's DevTools** (F12 or Cmd+Option+I) and check the Console tab. Click around the Reports page. What do you see?

You should have found at least 3 distinct categories of bugs just from visual inspection.

Step 2: Fix the bugs with Claude Code (~3-5 minutes)

Now paste this prompt into Claude Code:

```
The Reports page (/reports) has multiple bugs compared to the rest of the app.
I found that:
1. It doesn't translate to Japanese when I switch languages
2. It ignores the global filter bar
3. It spams the browser console with debug logs

Investigate the Reports page code, identify ALL the issues (there are more than
these 3), and fix them. Look at how other pages like Dashboard and Orders
are implemented for reference.
```

Claude will:

- Read `Reports.vue` and compare it with working pages like `Dashboard.vue`
- Identify all the bugs (there are 8+ issues)
- Refactor the component to match the patterns used in the rest of the app
- Fix the "Reports" nav tab translation in `App.vue`

Verify the fixes by refreshing `http://localhost:3000/reports` and checking:

- Language switching works
- Filters update the data
- No more `console.log` spam
- Code follows Composition API patterns

Stretch Goal

Fix the Backlog page (`/backlog`) — it has the same i18n problem. All its strings are hardcoded in English instead of using the `t()` translation function.



Go Further: Make It Yours

0 pts

You've covered the full Claude Code workflow: exploring a codebase, shipping features, writing tests, reviewing PRs, and integrating with GitHub. That's the foundation. Now it's time to go off-script.

Ideas to get you started

- **Add a new feature** — low-stock alerts, a search bar, an export-to-CSV button, a dark mode toggle. Ask Claude to implement it end-to-end.
- **Restyle the app** — change the colour palette, swap fonts, redesign the nav. Try: *"Redesign the app with a dark navy and orange theme."*
- **Improve the data model** — add supplier tracking, product categories, or a simple audit log. Let Claude write the migrations and wire up the UI.
- **Extend the tests** — ask Claude to increase coverage, add edge cases, or set up a CI workflow with GitHub Actions.
- **Break something on purpose** — introduce a bug and see how quickly Claude can find and fix it with a single prompt.

How to approach it

- Describe what you want in plain language. Claude will read the codebase, plan the changes, and implement them.
- Push back if the output isn't right — iterate just like you would with a teammate.

There's no wrong answer here. The goal is to get a feel for what Claude Code can do when you give it real, open-ended problems.

Go deeper: Check out the Claude Code best practices guide for tips on prompt patterns, `CLAUDE.md` setup, agentic workflows, and more.